extension methods include an overload with a `reloadOnChange` parameter. If this parameter is set to `true`, the app monitors the filesystem for changes to the file and triggers a complete rebuild of the `IConfiguration`, if needs be. The following listing shows how to add configuration reloading to the appsettings.json file added manually to the `ConfigurationManager`.

Listing 10.5 Reloading appsettings.json when the file changes

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Configuration.Sources.Clear();
builder.Configuration
    .AddJsonFile(
        "appsettings.json",
        optional: true,
        reloadOnChange: true);    ❶

WebApplication app = builder.Build();

app.MapGet("/", () => app.Configuration.AsEnumerable());

app.Run();
```

❶ IConfiguration will be rebuilt if the appsettings.json file changes.

Throughout section 10.2, you've seen how to customize the `ConfigurationManager` providers by clearing the default sources and adding your own, but in most cases, that won't be necessary. As described in section 10.2.1, the default providers added by `WebApplicationBuilder` are normally good enough unless you want to add a new provider, such as Azure Key Vault. As a bonus, `WebApplicationBuilder` configures the appsettings.json with `reloadOnChange:true` by default. It's worth sticking with the defaults initially and clear the sources and start again only if you really need to.

WARNING Adding a file configuration source using `reloadOnChange:true` isn't entirely free, as ASP.NET Core sets up a file watcher in the background. Normally, this situation isn't problematic, but if